Lecture 9 - Serializable Snapshot Isolation

Main paper for today: **Serializable Snapshot Isolation in PostgreSQL**

- Auxiliary papers:
  - **Making Snapshot Isolation Serializable**
    - Characterizing serialization graphs
  - **Serializable Isolation for Snapshot Databases**
    - Making SI -> SSI
- Q: thoughts on main paper?
- Personal commentary on paper:
  - A beautiful paper:
    - theory + engineering
    - history, background - an intuitive and self-contained read
  - A nice description of practical challenges in implementing CC protocols in a modern system
    - Specifically one we've read about!
  - Plus some nice theory that wraps up the story that began with Adya
    - The notion of a *dangerous structure* - and how it can be used to conservatively abort
  - Paper itself is mostly about engineering challenges that were addressed:
    - but some (quite) interesting theory that builds on Cahill/Fekete's work (additional readings)
  - Interesting history (and coauthors!) - the impact of anomalies in the real world

Intro, overview

- Postgres did not support serializability, just SI
  - This was also true for most other databases as well
  - "An acceptable compromise"
- Big question: *could snapshot isolation be made serializable?*
- Two papers by Cahill/Fekete and co. answered the question, with the main one being Serializable isolation for snapshot databases, TODS'2009, which is the origin of SSI
- This paper was an attempt at implementing this within Postgres
  - first such implementation in a production database
    - nontrivial to do so - needs to integrate with replication, 2PC, ...
    - a big concern that emerged was bounding the amt of memory usage
  - also first implementation in a snapshot-based database, as opposed to one with 2PL (e.g., MySQL)
    - since locking was present, could leverage this
    - in this case, needed to implement a lock manager.
      - only for SSI read-write dependencies
        - so not full-fledged, but also introduces new challenges
        - specifically, "registering" the fact that a txn is reading an item
      - this lock mgr + MVCC data helps detect conflicts
  - plus some improvements for read-only txns
    - extend SSI to do so
    - introduce notions of "*safe snapshots and deferrable transactions*"
      - here, eliminate overhead of SSI because we know SI anomalies can't happen
- Results:
  - More concurrency than 2PL, but also serializable
  - Perf cost of <7% relative to SI

Setup

- Postgres uses READ COMMITTED by default
  - Recall: for 2PL, done via short duration read locks
  - With Postgres' multi versioning, can be done without locking for reads
    - READ COMMITTED - a snapshot per statement
    - Visibility of tuples determined by xmin, xmax, ..
- Postgres also supported SI as strongest isolation level
  - Recall READ COMMITTED $\ll$ SI $\ll$ SERIALIZABLE
  - Again, can be done without read locks.
    - SI - snapshot fixed at txn start
  - Users requesting SERIALIZABLE got SI
    - Happens in many other databases too...
  - This actually led to an anomaly detected in the Wisconsin Court (explaining the coauthor)
    - Hard to catch - sometimes silent mistakes
      - Recall write skew? r1[x]...r2[y]...w1[y]...w2[x]...(c1 and c2 occur)

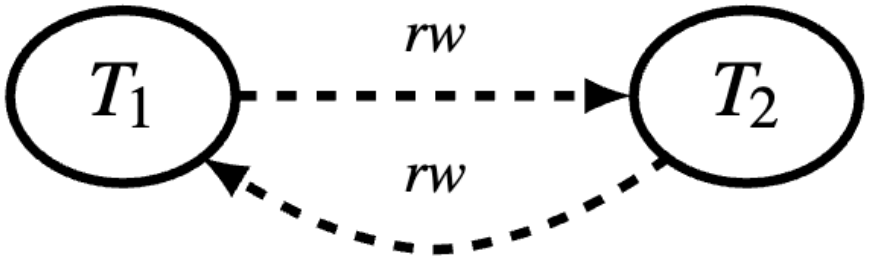
Snapshot Isolation

- Prevents dirty reads, phantoms, non-repeatable reads
- Hence was assumed to be like SERIALIZABLE
- Anomalies in SI:
  - Write Skew
    - example:

T_1	T_2
$x \leftarrow$ **SELECT** **COUNT**$(*)$ **FROM** doctors **WHERE** on−call = **true**	
	$x \leftarrow$ **SELECT** **COUNT**$(*)$ **FROM** doctors **WHERE** on−call = **true**
IF $x \geq 2$ **THEN** **UPDATE** doctors **SET** on−call = **false** **WHERE** name = Alice	
COMMIT	
	IF $x \geq 2$ **THEN** **UPDATE** doctors **SET** on−call = **false** **WHERE** name = Bob
.	**COMMIT**

time

    - Invariant: at least one doctor is on-call

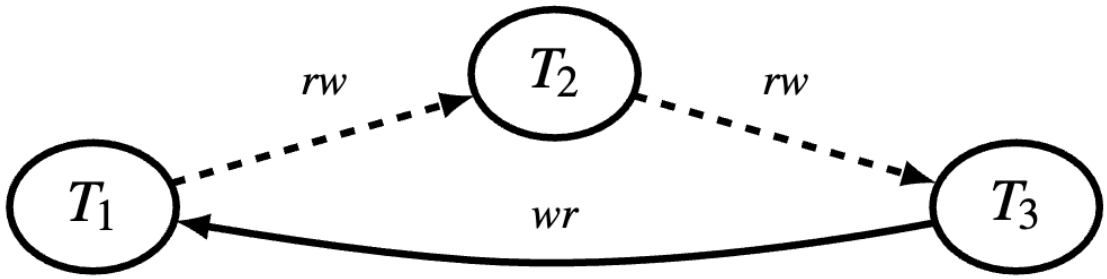


- T1's update is not visible to T2's read and vice-versa

- Batch Processing
  - Three txns, one is read-only

T_1 (REPORT)	T_2 (NEW-RECEIPT)	T_3 (CLOSE-BATCH)
	$x \leftarrow$ **SELECT** current_batch	
		INCREMENT current_batch
		COMMIT
$x \leftarrow$ **SELECT** current_batch		
SELECT SUM(amount) **FROM** receipts **WHERE** batch = $x - 1$		
COMMIT		
	INSERT INTO receipts **VALUES** (x, somedata)	
.	**COMMIT**	.

- Invariant: *after a REPORT transaction has shown the total for a particular batch, subsequent transactions cannot change that total.*
- Requires all three txns, even though T1 is read only.
  - Discovered by Fekete and co in 2004 (Paper: A read-only transaction anomaly under snapshot isolation)



- T3, which increments the batch number, appears to execute after T2, which reads the old version. The receipt inserted by T2 does not appear in T1's report, so T2 appears to execute after T1. Finally, T3 appears to execute before T1, completing the cycle (wr).
- Fekete's 2005 paper had several insights on the serialization graphs (from Adya) for SI. Several insights:
  - *Under SI, wr and ww edges are forward in commit order. The only edges that can go "backwards" are rw-anti-dependencies.*
  - *Every non-serializable SI history contains a cycle with at least two consecutive rw-anti-dependency edges.*
  - Cahill and Fekete's 2008 paper aims to use this to make SI serializable, next ...

Serializable Snapshot Isolation

- Why not implement serializability via strict 2PL (S2PL)?
  - Interesting history on Postgres: was originally 2PL (pre-1999), moved to MVCC
  - "*Users accustomed to the "readers don't block writers, and writers don't block readers" mantra would be surprised by the additional blocking, and a S2PL approach was unpalatable to most of the developer community*."
- Stronger statement from Fekete et al 2005 (Making snapshot isolation serializable):
  - **Theorem**: *For SI, every cycle in the serialization history graph contains a sequence of edges T1 -rw→ T2 -rw→ T3 where each edge is a rw-anti-dependency. **Furthermore, T3 must be the first transaction in the cycle to commit*.**
  - **Corollary**: *Transaction T1 is concurrent with T2, and T2 is concurrent with T3, because rw-anti-dependencies occur only between concurrent transactions.*
  - Go back and verify this holds for the two examples above!
- Adaptation to SI (SSI)
  - Track only anti-dependencies between txns
    - **Rule 1**: *If any transaction has both an incoming rw and an outgoing rw, SSI aborts one of the transactions involved.*
  - False positives, no false negatives
  - Claim: greater concurrency than OCC/S2PL
    - allows rw as long as they don't form a dangerous structure
    - Consider Batch Processing Example without T1 (read txn)
      - neither S2PL or OCC would allow this but SSI would
- Improvements (?):
  - Follow Rule 1, but only abort if T3 is first to abort
    - still could have false positives because the cycle may not exist
  - PSSI builds full serialization history graph and tests for cycles
    - tracks ww, wr in addition to rw
    - more overhead, but aborts less

Read-Only Txn Optimization

- **Read-only snapshot ordering optimization**
  - Reduces FP aborts

    - **Theorem 3**. *Every serialization anomaly contains a dangerous structure T1 -rw−→ T2 -rw−→ T3, where if T1 is read-only, T3 must have committed before T1 took its snapshot.*
        - **Proof**: If T1 is read-only, then its previous txn in cycle, T0 must have a wr edge to it (can't be ww or rw as T1 does not write). Thus, T1 must have included T0's changes in its snapshot and T0 must have committed before T1 did so. But since T3 commits first, T3 must have committed before T0 and before T1 took a snapshot
    - Said another way, for read only txn, *what matters is the time the snapshot was taken, any actions and when they were done is immaterial*
    - **Read-only snapshot ordering rule**: *if a dangerous structure is detected where T1 is read-only, it can be disregarded as a false positive unless T3 committed before T1's snapshot.*
- **Safe Snapshots**
    - **Definition**: "*A read-only transaction T has a safe snapshot if no concurrent read/write transaction has committed with a rw-anti-dependency out to a transaction that committed before T's snapshot, or has the possibility to do so*"
    - For each read txn, track concurrent txns, check if they have an rw anti-dependency out, wait until they commit (safely), then txn can proceed with fewer SSI checks/locks
- **Deferrable Txns**
    - Acquires a snapshot, waits for concurrent txns to finish - if there is any potential for dangerous structure, acquire a new snapshot.
    - Once all concurrent txns for the snapshots have finished, can start executing
    - This to me is like MVTO idea of picking an older version to read from (but a bit less stale - and have to wait more)

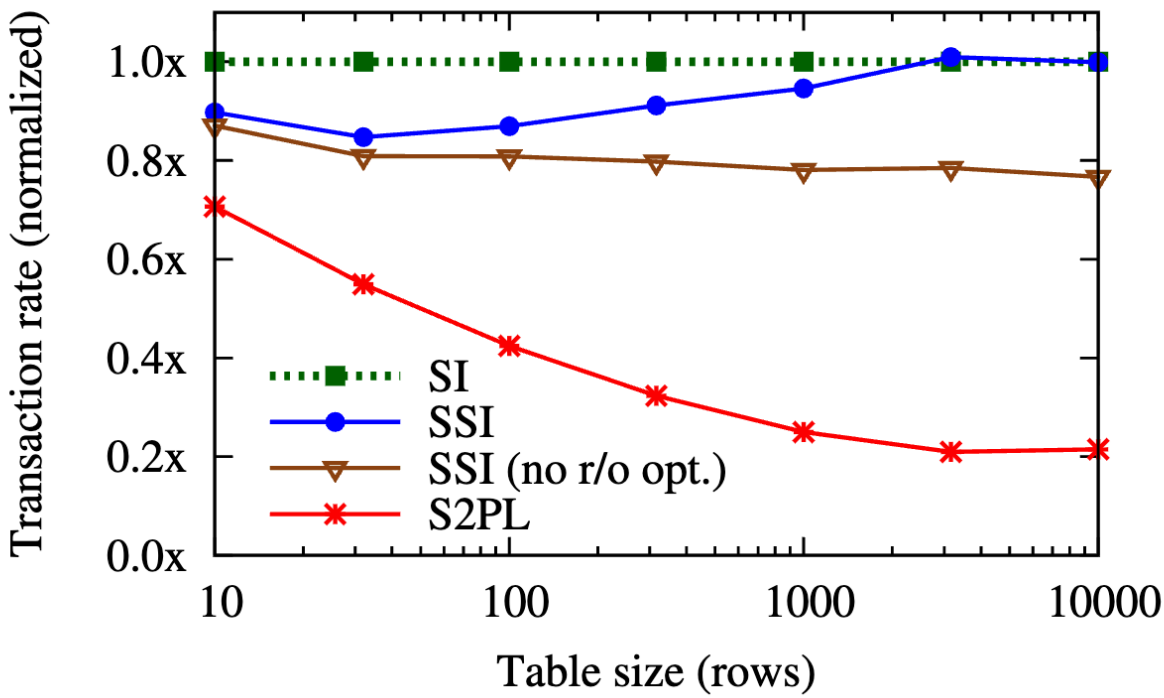
Implementation of the Core SSI Approach

- From SSI paper:
    - Transactions "acquire" a SIREAD lock on items they read; but:
        - This isn't a lock (it doesn't block writes)
        - This must persist until all concurrent txns commit
- SSI lock manager
    - Mainly for managing SIREAD "locks"
    - Uses index-range locking to allow for predicate locking
    - Or locking at relation level if needed
- **Detect** Need to detect rw anti-dependencies; two cases: w happens first chronologically or r happens first chronologically
    - If w happens first: can be done via MVCC data during the read
    - If r happens first: needs SIREAD locks to detect when performing a write
- **Track** Tracking the conflicts
    - rw anti-dependency pointers for each txn to the next
- **Resolve** Resolving conflicts
    - Abort T2 (when T2 is restarted, it won't be concurrent with T3); if T2 has committed, abort T1 (when T1 is restarted it won't be concurrent with either).

Memory Usage

- locks can't be released until the txn and all concurrent ones commit.
    - therefore "*single long-running transaction can easily prevent thousands of transactions from being cleaned up*"
- Employ safe snapshot ideas - safe snapshots don't need SIREAD locks or tracking of dependencies, same with deferrable txns.
- Aggressive cleanup
    - Per txn, track earliest committed txn to which it has a conflict out
    - If all remaining txns are read-only, can drop SIREADs - no new conflicts can form.
- Summarizing committed txns

Evaluation



-
- Microbenchmark - testing lots of tiny updates, and full-table scans

Takeaways

- SI eliminates many anomalies but not cycles involving only rw edges.
- All SI anomalies contain a rw-rw dangerous structure.
- SSI leverages this to bring "S" to SI
- Careful engineering + additional theory can make SSI pretty competitive!